Och nu nästa steg!

Eller, kanske en utveckling av förra steget…

Vi har fått det att rulla!

Vår Message går hela vägen vi har användare som är nöjda och allt är frid och fröjd.

Nu ska vi börja plocka ihop svar! Och bestämma vad vi vill ska skickas ut!

Tänk att vi har en lite större entity (hittills har vi haft typ en string och en int som mest)

{ ProduktName: String,
ProductDescription : String,
Price: Double,
Outgoing : boolean,
PurshasePrice: Double,
StockStatus: int,
Shelf: char
}

DETTA GÖRS I CONTROLLERN!

Vilken kan kännas ointuitivt!

MEN TÄNK SÅ HÄR:

Servicelagret ska inte veta något som controllerlagret.

Vi vill kunna ändra API:et, lägga till och skapa nya endpoints och versioner utan att ändra i Service och eller databasen.

Controller får datan från Service och sätter sedan ihop det till DTO:er som skickas vidare via SPRING.

Nu måste vi börja plocka ihop saker och ting och skicka tillbaka det som efterfrågas. Varken mer eller mindre.

Vi får en call get /product/info

Och vill skicka:

{ ProduktName: String,
ProductDescription : String,
Price: Double,
Shelf: char
}

REST BEHÖVER ETT OBJEKT FÖR ATT GÖRA EN JSON!
DET OBJEKTET KALLAR VI DTO (DATA TRANSFER OBJEKT).
DET ÄR BARA EN VANLIG JAVAKLASS MED GETTERS!

Eller hellre och modernare en record!

VI BYGGER EN DTO

{ ProduktName: String,
ProductDescription : String,
Price: Double,
Shelf: char
}

I praktiken lägger vi DTO:erna i ett nytt package som vil kallar dto eller liknande

Och de ser ut så här:

```
public record infoDto (Long id, String productName,
String productDescrition, Double price, char shelf){}
```

I praktiken i controllern

```
@GetMapping("/info/{id}")
public InfoDto getNumber(@PathVariable Long id) {

    Product product = service.getById(id);

    return new InfoDto (
product.getProductName,
product.getProductDescription,
product.getPrice(),
product.getShelf()
);
}
```

Vecka 3 Planering

Måndag:

Föreläsning versionering

Övningar/häfte ORM

(Eget arbete/repetition av databaser)

~12 Material för morgondagen

Tisdag:

Boilerroom:

Versionering och domänutbyggnad

Torsdag:

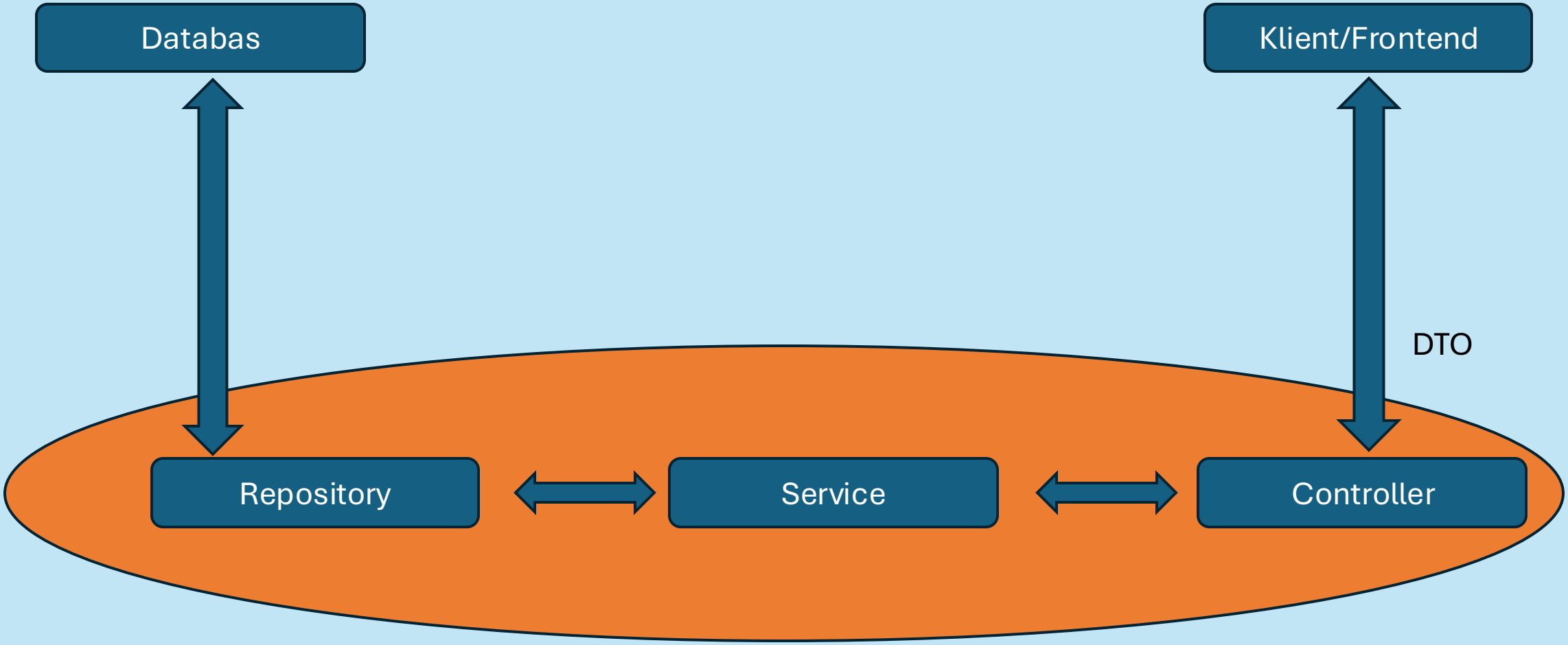
Testning/ Felhantering och/eller mer om versionering/ORM beroende på läget

ÖVNINGPROJEKTET - DAGS ATT HA EN STRUKTUR NU!

Versionering

What if saker och ting ändras!

Så här långt har vi:



Exempel: Vi har en webbutik där vår frontend använder det här anropet överallt:

GET /api/products/42

```
{
  "id": 42,
  "name": "Gaming Mouse",
  "price": 59.99
}
```

Men med tiden vill vi ändra modellen till vi vill lägga till ytterligare fält:

GET /api/products/42


```
{
  "id": 42,
  "ProductName": "Gaming Mouse",
  "price": 59.99,
  "currency": "USD",
  "inventory": 12
}
```

Om vi går in och ändrar i våra endpoints kan mängder av frontends kvadda!

GET /api/products/42

```
{
  "id": 42,
  "name": "Gaming Mouse",
  "price": 59.99
}
```

GET /api/products/42

```
{
  "id": 42,
  "ProductName": "Gaming Mouse",
  "price": 59.99,
  "currency": "USD",
  "inventory": 12
}
```

Framför allt om vi gör så kallade "Breaking changes"

Ändring	Breaking
ta bort fält	ja
ändra datatyp	ja
ändra endpoint	ja
lägga till fält	oftast nej

Lösningen är att versionerar
dvs vi ser till att gör olika
versioner av vår API.
Och vi ser till att strukturmässigt
ta höjd för att detta kommer att
hända.

I praktiken – vi gör en ny Controller och nya dto:er för den nya versionen

Vi gör nya dto:er för den nya versionen

Och vi lägger till ett v1,v2 etc i våra endpoints:

/api/v1/products
/api/v2/products

```
controller
├─ v1
│   └─ ProductControllerV1
│
└─ v2
    └─ ProductControllerV2

dto
├─ v1
│   └─ ProductDtoV1
│
└─ v2
    └─ ProductDtoV2
```

Mycket duplicerad kod här!

Varför?

```
service
└─ ProductService

Entity
└─ Product

repository
└─ ProductRepository
```

Men det här försöker vi så långt det bara går att hålla stabilt!

Versonering kortversionen:

1. Versionering sker i controllerlagret ***– inte i servicelagret!***
2. Vi gör en controller och nya dto:er för varje version - ÄVEN om det innebär duplicerad kod!
3. Vi tar tidigt höjd för versionering i strukturen – dvs vi lägger våra dto:er och vår kontroller i undermappar med med namn som v1, v2 etc.
4. Vi tar höjd för framtiden genom att ha med v1, v2 etc i våra endpoints: */api/v1/products*
5. Ny version skapas vid breaking changes - alltså när vi riskerar att förstöra för frontenden –dvs (vanligen) inte om vi bara lägger till ett fält!

Fråga:

```
{
  "id": 42,
  "name": "Gaming Mouse",
  "price": 59.99
}
```

```
{
 "id": 42,
"name": "Gaming Mouse",
"price": 59.99,
"inventory": 12
}
```

Behöver vi versoniera? (varför/ varför inte?)

Fråga:

```
{
  "id": 42,
  "name": "Gaming Mouse",
  "price": 59.99
}
```

```
{
  "id": 42,
  "name": "Gaming Mouse"
}
```

Behöver vi versoniera? (varför/ varför inte?)

Så... Viktigt är alltså!

Vi tar ALLTID höjd för att det kommer att komma ny versioner

Det betyder att vi ALLTID lägger DTO:er och Controllers i packages med namn v1/version1 etc.
Och vi låter våra endpoints innehålla versionnamnet.

Gör vi någon någon typ av ändringar av breaking changes så gör vi en ny version

Några ord om att fasa ut gamla versioner

En bra tumregel i verkligeheten är att aldrig har fler än två versioner.

1. Släpp v2
2. Markera v1 deprecated
3. 6 månader grace period
4. Returnera 410 Gone
6. Ta bort kod

Sammanfattning

Vi tar höjd för versionering genom att:

Ha en versionsstruktur för controllers och dto:s

Se till att versionsnummer finns med i våra endpoints

Hålla koll på när det dyker upp Breaking Changes

Informer och fasa in förändringarna på ett bra sätt

Minns att frontenden byggs av en tvåmeters psykopat som har din hemdadress!